Aula 11 - Seguranca de APIs e Mass Assignment

APIs recebem objetos estruturados e podem aceitar campos que o usuario nao deveria controlar. Mass Assignment ocorre quando o backend atualiza o modelo diretamente com o body sem selecionar campos permitidos.

Pontos-chave

- Use schemas de entrada.
- Separe DTO de entrada do modelo do banco.
- Campos como role, saldo e status exigem controle explicito.

Exemplo tecnico

Codigo / comando

```
const Schema = z.object({ name: z.string(), email: z.string().email() });
const data = Schema.parse(req.body);
```

Portugol - equivalente didatico

```
se nome = vazio ou emailInvalido(email) entao
    responder(400)
senao
    atualizar(nome, email)
fimse
```

Aula 12 - BOLA / IDOR em APIs

BOLA e Broken Object Level Authorization: a API identifica corretamente o objeto, mas falha ao decidir se o usuario atual pode acessa-lo. E um dos problemas centrais em APIs modernas.

Pontos-chave

- Autorizacao deve ser por objeto.
- UUID nao substitui autorizacao.
- Listagens e exportacoes tambem precisam de filtro por usuario/empresa.

Exemplo tecnico

Codigo / comando

```
const invoice = await db.invoices.findById(req.params.id);
if (invoice.companyId !== req.user.companyId) return res.sendStatus(403);
```

Portugol - equivalente didatico

```
fatura <- buscarFatura(id)
se fatura.empresaId <> usuarioAtual.empresaId entao
    responder(403)
fimse
```

Aula 13 - Upload de Arquivos

Upload pode introduzir arquivos inesperados, conteudo ativo, extensoes enganosas e consumo excessivo. A aplicacao deve limitar tamanho, tipos esperados, nomear arquivos com seguranca e armazenar de forma apropriada.

Pontos-chave

- Nao confie apenas na extensao.
- Defina limite de tamanho e quantidade.
- Evite servir upload executavel como codigo.

Exemplo tecnico

Codigo / comando

```
if (file.size > 5_000_000) return res.status(400).send("Arquivo grande");
if (!ALLOWED_TYPES.includes(file.mimetype)) return res.status(400).send("Tipo invalido");
```

Portugol - equivalente didatico

```
se tamanhoArquivo > 5000000 entao responder(400) fimse
se tipoArquivo nao pertence tiposPermitidos entao responder(400) fimse
```

Aula 14 - CORS

CORS e uma politica do navegador que controla quais origens podem ler respostas entre sites. Nao e autenticacao. Uma configuracao permissiva pode expor dados quando combinada com credenciais e endpoints sensiveis.

Pontos-chave

- Origin deve ser validada conforme necessidade real.
- Nao reflita qualquer Origin cegamente.
- CORS nao protege chamadas servidor-servidor.

Exemplo tecnico

Codigo / comando

```
const permitidas = new Set(["https://app.exemplo.com"]);
if (!permitidas.has(req.headers.origin)) return res.sendStatus(403);
```

Portugol - equivalente didatico

```
se origem nao pertence origensPermitidas entao
    responder(403)
fimse
```

Aula 15 - SSRF - Server-Side Request Forgery

SSRF acontece quando o servidor faz uma requisicao para um destino controlado pelo usuario sem restricao suficiente. No laboratorio, uma rota /preview conseguia acessar um servidor interno. A defesa aplicada foi allowlist de hosts.

Pontos-chave

- O erro estava no fetch(url) com URL nao validada.
- Allowlist e mais segura que blacklist simples.
- Erros de entrada podem usar 400; falhas internas reais podem usar 500.

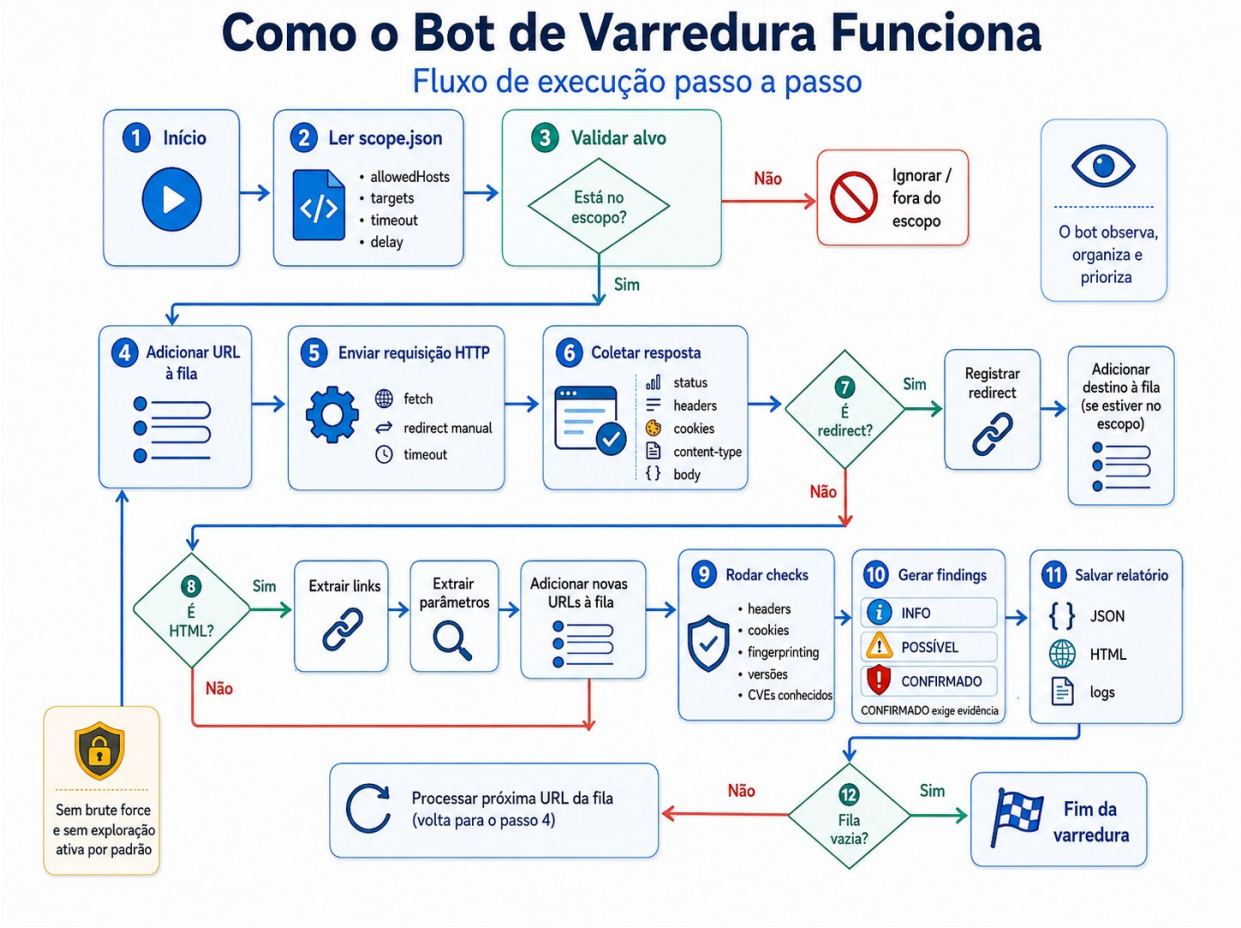
Exemplo tecnico

Codigo / comando

```
const u = new URL(req.query.url);
const permitidos = ["example.com"];
if (!permitidos.includes(u.hostname)) return res.status(400).send("Destino proibido");
const r = await fetch(u);
```

Portugol - equivalente didatico

```
url <- analisarURL(urlRecebida)
se host(url) nao pertence permitidos entao
    responder(400)
senao
    resposta <- requisitar(url)
fimse
```



O scanner posterior reutiliza os mesmos conceitos de requisicao, resposta e escopo.

Bloco 2 - Logica de negocio, tokens e OAuth

Aula 16 - Business Logic Vulnerabilities

Falhas de logica de negocio surgem quando o fluxo tecnicamente funciona, mas as regras permitem estados ou combinacoes que nao deveriam existir: quantidade negativa, cupom reutilizado, preco enviado pelo cliente ou etapa pulada.

Pontos-chave

- Teste zero, negativo, repeticao e ordem das etapas.
- Valores economicos devem ser calculados no servidor.
- Estado atual deve autorizar a proxima transicao.

Exemplo tecnico

Codigo / comando

```
if (!Number.isInteger(qtd) || qtd < 1) return res.status(400).send("Quantidade invalida");
const produto = await db.produtos.findById(produtoId);
const total = produto.preco * qtd;
```

Portugol - equivalente didatico

```
se quantidade < 1 entao responder(400) fimse
produto <- buscarProduto(produtoId)
total <- produto.preco * quantidade
```

Aula 17 - Race Conditions

Uma race condition ocorre quando operacoes corretas isoladamente produzem estado incorreto quando executadas ao mesmo tempo. Saldos, estoque, cupons e reservas sao exemplos tipicos.

Pontos-chave

- Check-then-act sem exclusao pode falhar.
- Transacoes e bloqueios do banco ajudam.
- Idempotencia e restricoes unicas tambem sao importantes.

Exemplo tecnico

Codigo / comando

```
await db.query("BEGIN");
const saldo = await db.query("SELECT saldo FROM contas WHERE id=$1 FOR UPDATE", [id]);
// validar e atualizar
await db.query("COMMIT");
```

Portugol - equivalente didatico

```
iniciarTransacao()
bloquearConta(id)
saldo <- lerSaldo(id)
se saldo >= valor entao atualizarSaldo(id, saldo-valor) fimse
confirmarTransacao()
```

Aula 18 - Privilege Escalation

Escalada vertical ocorre quando um usuario consegue privilegios maiores; horizontal quando acessa recursos de outro usuario no mesmo nivel. O cliente nunca deve decidir seu proprio papel.

Pontos-chave

- Rotas administrativas precisam de verificacao explicita.
- Self-service nao deve aceitar role.
- Permissoes precisam ser aplicadas em cada acao sensivel.

Exemplo tecnico

Codigo / comando

```
const data = { name: req.body.name, email: req.body.email };
await db.users.update(req.user.id, data);
```

Portugol - equivalente didatico

```
dados.nome <- nomeRecebido
dados.email <- emailRecebido
// papel nao e aceito aqui
atualizarUsuario(usuarioAtual.id, dados)
```

Aula 19 - JWT Avancado, Sessoes e Refresh Tokens

Access tokens devem ser curtos; refresh tokens podem durar mais e precisam ser protegidos e revogaveis. jwt.decode apenas le; jwt.verify valida assinatura e claims conforme configuracao.

Pontos-chave

- Separe segredos/chaves e finalidades.
- Valide exp, issuer e audience quando aplicavel.
- Logout forte pode exigir revogacao/rotacao do refresh token.

Exemplo tecnico

Codigo / comando

```
const access = jwt.sign({ sub: user.id }, ACCESS_SECRET, { expiresIn: "15m" });
const payload = jwt.verify(access, ACCESS_SECRET);
```

Portugol - equivalente didatico

```
token <- assinarToken(usuario.id, 15_minutos)
dados <- verificarToken(token, SEGREDO_ACESSO)
```

Aula 20 - OAuth 2.0, OpenID Connect, State e PKCE

OAuth e principalmente delegacao de autorizacao; OpenID Connect adiciona identidade. No Authorization Code Flow, a aplicacao recebe um codigo e troca por tokens. state liga resposta ao fluxo iniciado; PKCE protege contra reutilizacao de codigo interceptado.

Pontos-chave

- Redirect URI deve ser estrita.
- state deve ser imprevisivel e verificado.
- OIDC exige validacao correta de issuer, audience e assinatura.

Exemplo tecnico

Codigo / comando

```
req.session.oauthState = crypto.randomUUID();
// callback
if (req.query.state !== req.session.oauthState) return res.sendStatus(400);
```

Portugol - equivalente didatico

```
estadoSessao <- gerarUUID()
se estadoRecebido <> estadoSessao entao
    responder(400)
fimse
```